Problem Statement 1: AutoQA Pro: End-to-End Test Automation Framework for an E-Commerce Web Application

1. Hackathon Objective

This hackathon evaluates students on:

- *Selenium WebDriver fundamentals and browser automation*
- *Page Object Model (POM) design pattern*
- *TestNG framework – annotations, listeners, and DataProviders*
- *Explicit wait strategy for handling dynamic elements*
- *Screenshot capture on test failure*
- *Clean, reusable, and maintainable framework design*

2. Problem Statement

Design and develop a ***Selenium-Java Automation Test Framework for an E-Commerce web application.***

The framework should automate end-to-end test scenarios across the following functional areas:

- *User login, registration, and logout*
- *Product search and detail verification*
- *Shopping cart – add and remove items*
- *Form validations – empty fields and invalid inputs*

The framework must follow the Page Object Model pattern and must be structured as a reusable framework — not just a collection of standalone test scripts.

3. Application Under Test (AUT)

URL: https://automationexercise.com

GUVI

© 2025 GUVI GEEK NETWORK PVT LTD. All rights reserved. No part of this document may be reproduced or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without prior permission from GUVI.

Confidential

No backend setup required. Students directly automate this live site.

Key modules available for testing:

- ***Auth*** *– Register, Login, Logout*
- ***Products*** *– Search, Browse by category, View product details*
- ***Cart*** *– Add, Remove items*
- ***Checkout*** *– Address, Payment, Order confirmation*

3. Test Modules and Scope

3.1 Test Module 1 – User Authentication

- *Verify successful login with valid credentials*
- *Verify login failure with wrong password and validate the error message*
- *Verify logout redirects to the home page*
- *Verify registration with a new unique user*

3.2 Test Module 2 – Product Search and Browse

- *Search for a product by keyword and verify results are displayed*
- *Navigate to a category and verify products are listed*
- *Open a product detail page and verify name and price are shown*

3.3 Test Module 3 – Shopping Cart

- *Add a product to cart and verify it appears with correct name and price*
- *Remove a product from cart and verify it no longer appears*
- *Verify cart count updates when a product is added*

3.4 Test Module 4 – Checkout Flow

- *Complete checkout from cart to order confirmation*
- *Attempt checkout without login and verify redirect to login page*

Confidential

© 2025 GUVI GEEK NETWORK PVT LTD. All rights reserved. No part of this document may be reproduced or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without prior permission from GUVI.

3.5 Test Module 5 – Form Validations

- *Submit registration form with empty fields and verify error messages*
- *Enter invalid email format and verify validation is triggered*

4. Must-Have Requirements

4.1 Page Object Model (POM)

- *Every page must have a dedicated Page class (LoginPage, ProductPage, CartPage)*
- *Locators and action methods must be inside Page classes only*
- *Test classes must only call Page class methods — no findElement or click in test classes*
- *A BasePage class must have at least one shared utility (example: waitForElement)*

4.2 TestNG Configuration

- *testng.xml must define the test suite and list all test classes*
- *Use @BeforeMethod and @AfterMethod for browser launch and teardown*
- *Use at least one @DataProvider for login tests (valid and invalid credentials)*

4.3 WebDriver Management

- *Use WebDriverManager — no manual driver setup*
- *Browser name must be read from config.properties (chrome or firefox)*

4.4 Wait Strategy

- *Thread.sleep() is strictly not allowed — deduction will be applied*
- *Use WebDriverWait with ExpectedConditions for elements that load dynamically*

4.5 Configuration Management

- *config.properties must hold: browser, base URL, and timeout value*
- *A ConfigReader class must read and provide these values*
- *No hardcoded URLs or values in any test or page class*

Confidential

© 2025 GUVI GEEK NETWORK PVT LTD. All rights reserved. No part of this document may be reproduced or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without prior permission from GUVI.

4.6 Screenshot on Failure

- *Implement ITestListener to capture a screenshot automatically when a test fails*
- *Save screenshots with a timestamp in the filename inside a /screenshots/ folder*

4.7 Test Reporting

- *ExtentReports must generate an HTML report after every run*
- *Report must show test name, pass/fail status, and screenshot on failure*

Required Maven dependencies:

- *selenium-java*
- *testng*
- *webdrivermanager*
- *extentreports*

5. Business Rules (Strictly Enforced)

- *Thread.sleep() anywhere results in a score deduction*
- *Hardcoded URLs or credentials in any class other than config.properties results in a deduction*
- *WebDriver code directly inside test methods results in a POM score deduction*
- *Suite must run from testng.xml using: mvn test*

6. Good to Have

- *@DataProvider reading from a JSON or Excel file instead of inline data*
- *Parallel execution in testng.xml (thread-count="2")*
- *FluentWait for slow-loading elements*
- *Headless browser mode from config.properties*
- *Retry on failure using IRetryAnalyzer*

Confidential

© 2025 GUVI GEEK NETWORK PVT LTD. All rights reserved. No part of this document may be reproduced or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without prior permission from GUVI.

7. Non-Functional Expectations

- *Clean separation between Page classes, Test classes, and config*
- *No duplicate code — shared actions must go into BasePage or utilities*
- *Meaningful method and variable names throughout*
- *No Thread.sleep() — all timing handled via WebDriverWait*
- *Code readability over quantity — a clean small framework beats a messy large one*

8. Hackathon Guidelines

- *Spend the first 20–30 minutes planning before writing any code*
- *Internet and AI usage is allowed — but every member must explain their own code*
- *Every member must contribute at least one Page class and one Test class*
- *No hardcoded URLs or credentials anywhere except config.properties*
- *Final walkthrough: 5 minutes per team, each member speaks about their own work*

9. Evaluation Criteria

- *Framework Design*
- *Test Coverage*
- *Selenium Proficiency*
- *Data-Driven and Config Design*
- *Reporting and Evidence*
- *Team Collaboration*

Confidential

© 2025 GUVI GEEK NETWORK PVT LTD. All rights reserved. No part of this document may be reproduced or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without prior permission from GUVI.